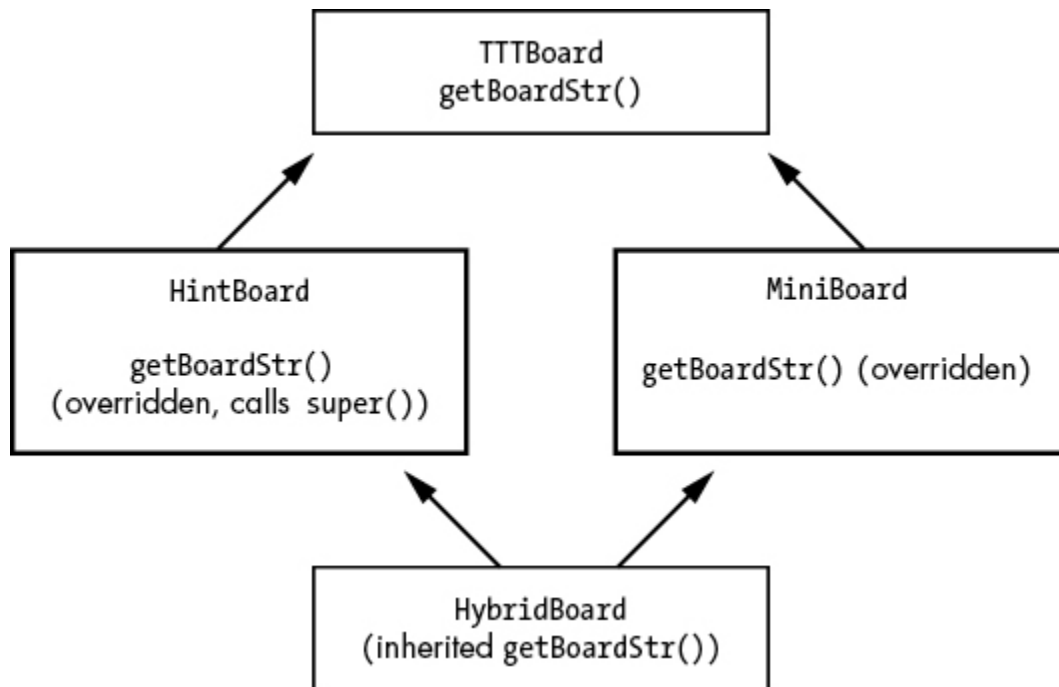




[Figure 16-2:](#) The four classes in our tic-tac-toe board program

When we call `getBoardStr()` on a `HybridBoard` object, Python knows that the `HybridBoard` class doesn't have a method with this name, so it checks its parent class. But the class has two parent classes, both of which have a `getBoardStr()` method. Which one gets called?

You can find out by checking the `HybridBoard` class's MRO, which is the ordered list of classes that Python checks when inheriting methods or when a method calls the `super()` function. You can see the `HybridBoard` class's MRO by calling its `mro()` method in the interactive shell:

---

```
>>> from tictactoe_oop import *
>>> HybridBoard.mro()
[<class 'tictactoe_oop.HybridBoard'>, <class 'tictactoe_oop.HintBoard'>, <class 'tictactoe_oop.MiniBoard'>, <class 'tictactoe_oop.TTTBoard'>, <class 'object'>]
```

---

From this return value, you can see that when a method is called on `HybridBoard`, Python first checks for it in the `HybridBoard` class. If it's not there, Python checks the `HintBoard` class, then the `MiniBoard` class, and finally the `TTTBoard` class. At the end of every MRO list is the built-in `object` class, which is the parent class of all classes in Python.